

Name: R. Manogna **BITSID:** 202217B3802

COMPUTER NETWORKS LABORATORY

Problem 1 — TCP Campus Announcement System

1. Problem Statement 1 — Implementation

Objective: Implement a TCP-based campus announcement system in which multiple clients can connect to a central server. Authorized faculty users can broadcast announcements, while student users can only receive announcements. The server handles multiple clients concurrently and displays join/leave events.

2. Program Design

The system follows a client-server architecture. The server listens on TCP port 5000 and accepts incoming connections. For every connected client, the server creates a separate thread, allowing multiple users to remain connected at the same time.

Main Components

Component	Purpose
TCP Server	Accepts clients, registers usernames, manages connections and broadcasts messages.
TCP Client	Connects to the server, registers a username and receives messages.
Authorization	faculty1 and admin can send announcements; other users are students.
Broadcast	Sends an announcement to all currently connected clients.
Threading	Creates one handler thread per client for concurrent communication.

3. TCP Protocol Used

TCP is a connection-oriented and reliable transport-layer protocol. The client establishes a TCP connection using the server IP address and port. After registration, data is exchanged using `sendall()` and `recv()`. TCP provides ordered and reliable delivery of the byte stream, making it appropriate for an announcement system where messages should not be silently lost or delivered out of order.

Message Flow

1. Server binds to 0.0.0.0:5000 and starts listening.
2. Client connects to 127.0.0.1:5000.
3. Server requests a username and returns REGISTERED.
4. Server announces the user's join event to other clients.
5. Authorized users send ANNOUNCE <message>.
6. Server timestamps and broadcasts the announcement to all connected clients.
7. On disconnect, the server broadcasts a system leave message.

4. Authorization Logic

The server maintains `AUTHORIZED_USERS = {'faculty1', 'admin'}`. If a client sends an ANNOUNCE command and its username is not in this set, the server returns an authorization error. The client also prevents student users from sending announcements. This provides client-side guidance and server-side enforcement.

5. Source Code — tcp_server.py

The server implements TCP socket communication, multi-client threading, authorization, broadcasting, timestamps, and connection cleanup.

```
import socket
import threading
from datetime import datetime

HOST = "0.0.0.0"
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen()
clients = {} # socket -> username
# Authorized faculty usernames
AUTHORIZED_USERS = {"faculty1", "admin"}

def broadcast(message, exclude_socket=None):
    for client_socket in list(clients.keys()):
        if client_socket != exclude_socket:
            try:
                client_socket.sendall(message.encode())
            except:
                remove_client(client_socket)

def remove_client(client_socket):
    if client_socket in clients:
        username = clients[client_socket]
        del clients[client_socket]

    try:
        client_socket.close()
    except:
        pass

    time = datetime.now().strftime("%H:%M:%S")
    message = f"[{time}] SYSTEM: {username} left the campus system.\n"

    print(message.strip())
    broadcast(message)

def handle_client(client_socket, address):
    try:
        client_socket.sendall(
            b"Enter username:\n"
        )
        username = client_socket.recv(1024).decode().strip()
        if not username:
            client_socket.close()
            return
        if username in clients.values():
            client_socket.sendall(
                b"Username already exists. Connection closed.\n"
            )
            client_socket.close()
            return
        clients[client_socket] = username

        time = datetime.now().strftime("%H:%M:%S")
        print(f"[{time}] {username} connected from {address}")

        client_socket.sendall(
            b"REGISTERED\n"
        )

    join_message = (
        f"[{time}] SYSTEM: {username} joined the campus system.\n"
    )

    broadcast(join_message, client_socket)
```

```

while True:
    data = client_socket.recv(1024)

    if not data:
        break

    message = data.decode().strip()

    # Announcement format:
    # ANNOUNCE <message>
    if message.startswith("ANNOUNCE "):

        if username not in AUTHORIZED_USERS:
            response = "ERROR: You are not authorized to send announcements.\n"
            client_socket.sendall(response.encode())
            continue
        announcement = message[9:].strip()
        if not announcement:
            continue

        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        final_message = (
            f"[{timestamp}] ANNOUNCEMENT from {username}: "
            f"{announcement}\n"
        )

        print(final_message.strip())

        broadcast(final_message)

    else:
        client_socket.sendall(
            b"Use: ANNOUNCE <message>\n"
        )

except Exception as e:
    print("Client error:", e)

finally:
    remove_client(client_socket)
print(f"TCP Announcement Server running on {HOST}:{PORT}")

while True:
    client_socket, address = server.accept()

    thread = threading.Thread(
        target=handle_client,
        args=(client_socket, address),
        daemon=True
    )

    thread.start()

```

6. Source Code — tcp_client.py

The client connects to the TCP server, registers the username, starts a background receiver thread, displays announcements, and restricts announcement sending to authorized users.

```
import socket
import threading

SERVER_IP = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

def receive_messages():
    while True:
        try:
            data = client.recv(4096)

            if not data:
                print("Disconnected from server.")
                break

            print("\n" + data.decode(), end="")

        except:
            break

client.connect((SERVER_IP, PORT))

print(client.recv(1024).decode(), end="")

username = input("Username: ")

client.sendall(username.encode())

response = client.recv(1024).decode()

print(response)

if "REGISTERED" not in response:
    client.close()
    exit()

thread = threading.Thread(
    target=receive_messages,
    daemon=True
)
thread.start()
print("Connected to campus announcement system.")

if username in {"faculty1", "admin"}:
    print("You are authorized.")
    print("Use: ANNOUNCE <message>")
else:
    print("You are a student.")
    print("You can only receive announcements.")
while True:
    try:
        message = input("> ")

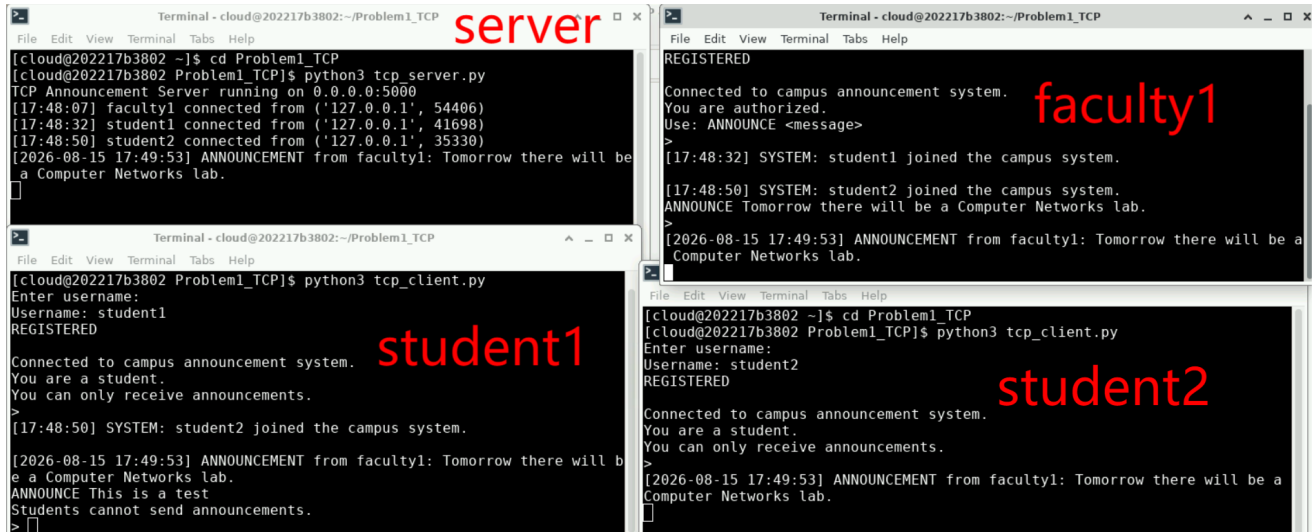
        if not message:
            continue
        if username not in {"faculty1", "admin"}:
            print("Students cannot send announcements.")
            continue

        client.sendall(message.encode())

    except KeyboardInterrupt:
        print("\nClosing client...")
        client.close()
        break
```

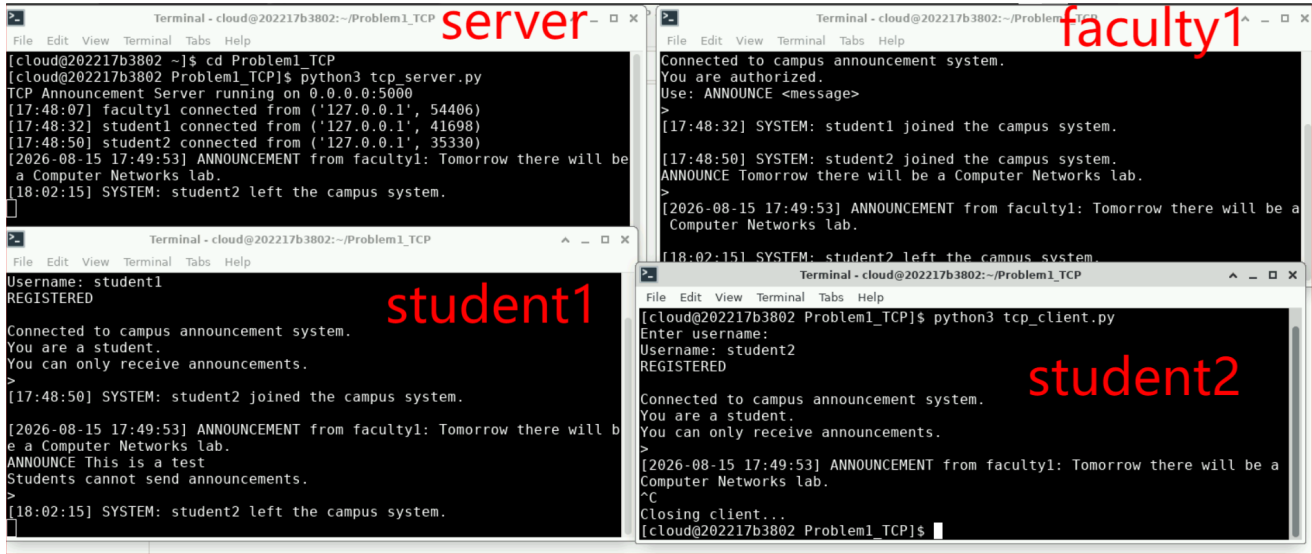
7. Compilation and Execution Evidence

The program was executed using Python 3 from the Problem1_TCP directory. The server was started first, followed by multiple TCP clients. The screenshots demonstrate successful registration, simultaneous clients, announcement broadcasting, authorization behavior, and client disconnection.



The figure consists of four terminal window screenshots arranged in a 2x2 grid. The top-left window, labeled 'server' in red, shows the server running and receiving connections from 'faculty1', 'student1', and 'student2'. It also shows an announcement from 'faculty1' at 17:49:53. The top-right window, labeled 'faculty1' in red, shows the faculty user registering and receiving the announcement. The bottom-left window, labeled 'student1' in red, shows the student registering and receiving the announcement. The bottom-right window, labeled 'student2' in red, shows the student registering and receiving the announcement. All windows show the system messages for client connections and announcements.

Figure 1. Multiple clients receive the same faculty announcement; student attempts to announce are rejected.



The figure consists of four terminal window screenshots arranged in a 2x2 grid. The top-left window, labeled 'server' in red, shows the server running and receiving connections from 'faculty1', 'student1', and 'student2'. It also shows an announcement from 'faculty1' at 17:49:53 and a system message at 18:02:15 stating 'student2 left the campus system.' The top-right window, labeled 'faculty1' in red, shows the faculty user registering and receiving the announcement. The bottom-left window, labeled 'student1' in red, shows the student registering and receiving the announcement. The bottom-right window, labeled 'student2' in red, shows the student registering and receiving the announcement, followed by a 'Closing client...' message at 18:02:15. All windows show the system messages for client connections and announcements.

Figure 2. Client disconnection is reflected by the server SYSTEM leave message.

8. Functionalities Demonstrated

Functionality	Observed Result
Server startup	TCP Announcement Server running on 0.0.0.0:5000.
Client registration	Clients receive REGISTERED after submitting a username.
Multiple clients	faculty1, student1 and student2 can remain connected concurrently.
Faculty authorization	faculty1 can use ANNOUNCE <message>.
Student restriction	Student client reports that students cannot send announcements.
Broadcast	Faculty announcement is received by the connected clients.
Join notification	SYSTEM message is generated when a user joins.
Leave notification	SYSTEM message is generated when a user disconnects.
Timestamping	Announcements and system events include timestamps.

9. Conclusion

Problem 1 was implemented successfully using Python TCP sockets and multithreading. The execution evidence shows that multiple clients can connect to the same server, authorized faculty can broadcast announcements, students can receive them, unauthorized announcement attempts are rejected, and connection events are reported to the system.